Module-12c Linear filtering with DFT

During the course of our study we have found that the output of an LTI system with impulse response $h[n]$ is given by convolution of $h[n]$ with the input $x[n]$, i.e., $y[n] = x[n] * h[n]$. Recall that if we consider the DTFT of $h[n]$ and $x[n]$ to be $H(e^{j\omega})$ and $X(e^{j\omega})$, respectively, then the frequency response of the output is given by $Y(e^{j\omega}) = H(e^{j\omega})X(e^{j\omega})$. Thus, an LTI system changes the frequency spectrum of the input. This process is also known in the literature as *linear filtering*. This is what we have been doing from almost the start of our classes. We are just giving it a name now – linear filtering. In short, Linear filtering is a process in which part of the signal frequency spectrum is modified by the transfer function of the filter (LTI system).

Note that computing $H(e^{j\omega})$ and $X(e^{j\omega})$ is not possible in a computer, because these are continuous functions of ω. One idea can be that we sample $X(e^{j\omega})$ and $H(e^{j\omega})$, i.e., find the DFT of $x[n]$ and $h[n]$, respectively at N-equally spaced frequencies. Let the DFTs of $x[n]$ and $h[n]$ be $X[k]$ and $H[k]$. Note that IDFT of $X[k]H[k]$ will give us the circular convolution of $x[n]$ and $h[n]$, i.e., $x[n] \otimes h[n]$. However, the output $y[n]$ is the linear convolution of $x[n]$ and $y[n]$, i.e., $x[n] * h[n]$. So, IDFT of $X[k]H[k]$ would not help us in getting the output. So, the straight forward question is: Can we use DFT to compute linear convolution? Let us explore this.

1 Linear filtering

Suppose that we have a finite-duration sequence $x[n]$ of length L and the impulse response of the system is of length M. Without loss of generality, let

$$x[n] = 0, \qquad n < 0 \text{ and } n \geqslant L, \qquad h[n] = 0, \qquad n < 0 \text{ and } m \geqslant M.$$

The output sequence is then given by

$$y[n] = h[n] * x[n] = \sum_{k=0}^{M-1} h[k]x[n-k]. \tag{1}$$

Since $h[n]$ and $x[n]$ are finite-duration sequences, their convolution is also finite-duration. Further, the duration of $y[n]$ is $L + M - 1$ (Recall you established this relation in Problem 6a of Assignment-1).

In the frequency domain, we have

$$Y(e^{j\omega}) = X(e^{j\omega})H(e^{j\omega}). \tag{2}$$

If the $(L+M-1)$ duration sequence $y[n]$ has to be uniquely represented in the frequency domain by samples of $Y(e^{j\omega})$, then the number of samples must equal or exceed $L+M-1$. Therefore, a DFT of size $N \geqslant L+M-1$, is required to represent $y[n]$ in the frequency domain (see Frequency Domain Sampling Module-12a for minimum number of samples required for perfect reconstruction). Now for $k = 0, 1, \cdots, N-1$, $Y[k] = Y(e^{j\omega})|_{\omega=\frac{2\pi}{N}k} = X(e^{j\omega})H(e^{j\omega})|_{\omega=\frac{2\pi}{N}k}$. Then

$$Y[k] = X[k]H[k] \qquad k = 0, 1, \cdots, N-1, \tag{3}$$

where $X[k]$ and $Y[k]$ are N-point DFTs of the corresponding sequences $x[n]$ and $h[n]$ with $N \geqslant L+M-1$. Since the sequences $x[n]$ and $h[n]$ have a duration less than N, we simply pad these sequences with zeros to increase their length to N. This increase in the size of the sequences $x[n]$ and $h[n]$ does not alter their spectra $X(e^{j\omega})$ and $Y(e^{j\omega})$.

Note that in order to uniquely reconstruct $x[n]$ from $X(e^{j\omega})$, we can sample $X(e^{j\omega})$ at the frequencies $\omega_k = \frac{2\pi k}{L}$, $k = 0, 1, \ldots, L-1$. However, now we are sampling $X(e^{j\omega})$ at $\Omega_k = \frac{2\pi k}{N}$, $k = 0, 1, \ldots, N-1$. Since $N \geq L$, we have more number of samples of $X(e^{j\omega})$ if we sample at Ω_k compared to sampling at ω_k. A similar argument holds for $H(e^{j\omega})$ as well, since $N \geq M$.

Since $N = (L+M-1)$-point DFT of $y[n]$ is sufficient to represent $y[n]$ in the frequency domain, it follows that the multiplication of the N-point DFTs $X[k]$ and $H[k]$, by equation (3), followed by computation of N-point IDFT must yield the sequence $y[n]$.

Therefore, linear convolution of $x[n]$ with $h[n]$ can be accomplished using circular convolution, if the length of the sequences $x[n]$ and $h[n]$ are increased to N by zero padding. We illustrate this with the help of an example.

Example 1.1 *Consider the signals $x_1[n]$ and $x_2[n]$ given in Figure 1.*

We want to find the linear convolution of the two signals using circular convolution.
In order to compute the circular convolution, we look at each sequence as points on circles. Then using folding and rotation techniques, we compute the final result of the convolution. Refer to module-12b for a detailed explanation of how to find circular convolution. Thus, $x_1[n] * x_2[n] =: x_3[n]$ is given by

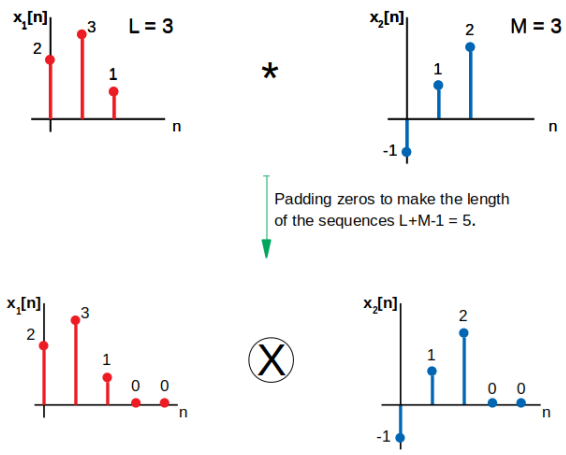
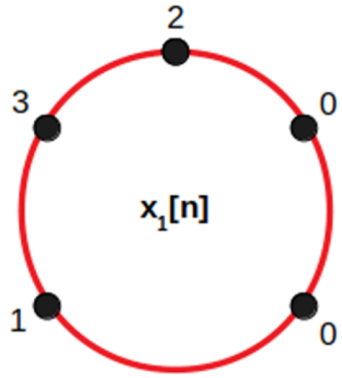
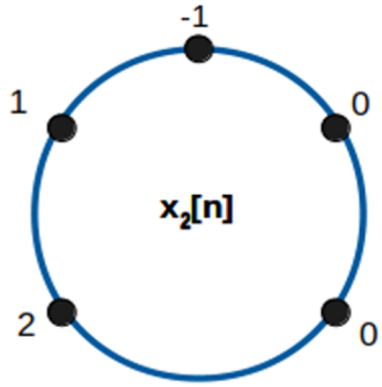
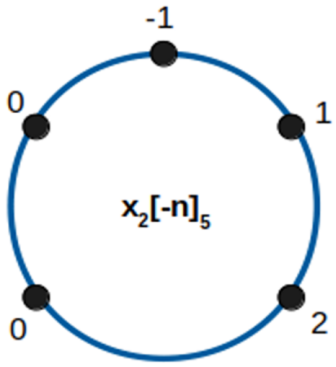


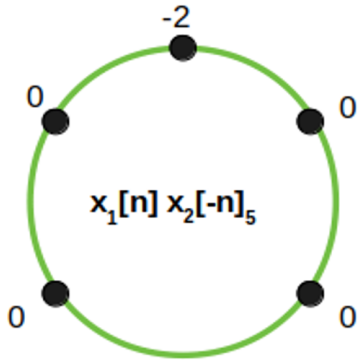
Figure 1: Example 1.1





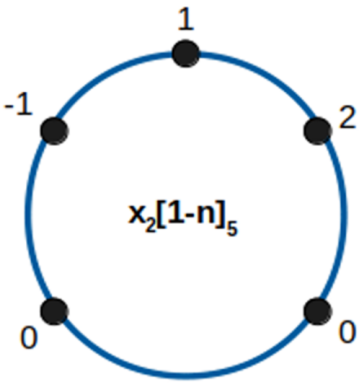


Folded sequence

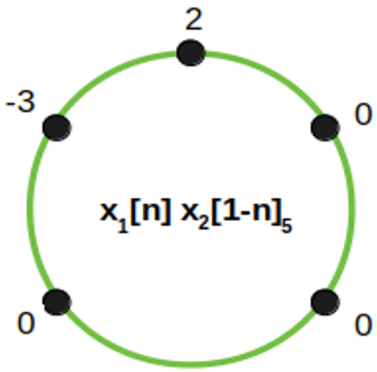


Product sequence

Sum = -2

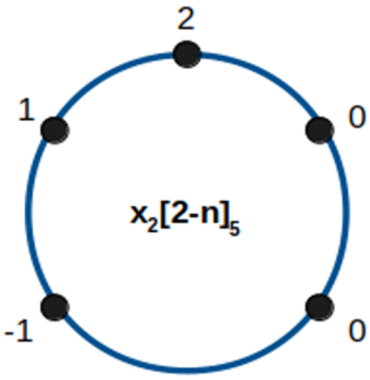


Folded sequence with one
unit of anticlockwise rotation

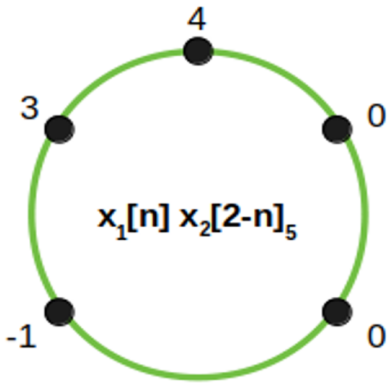


Product sequence

Sum = -1



Folded sequence with two
units of anticlockwise rotation



Product sequence

Sum = 6

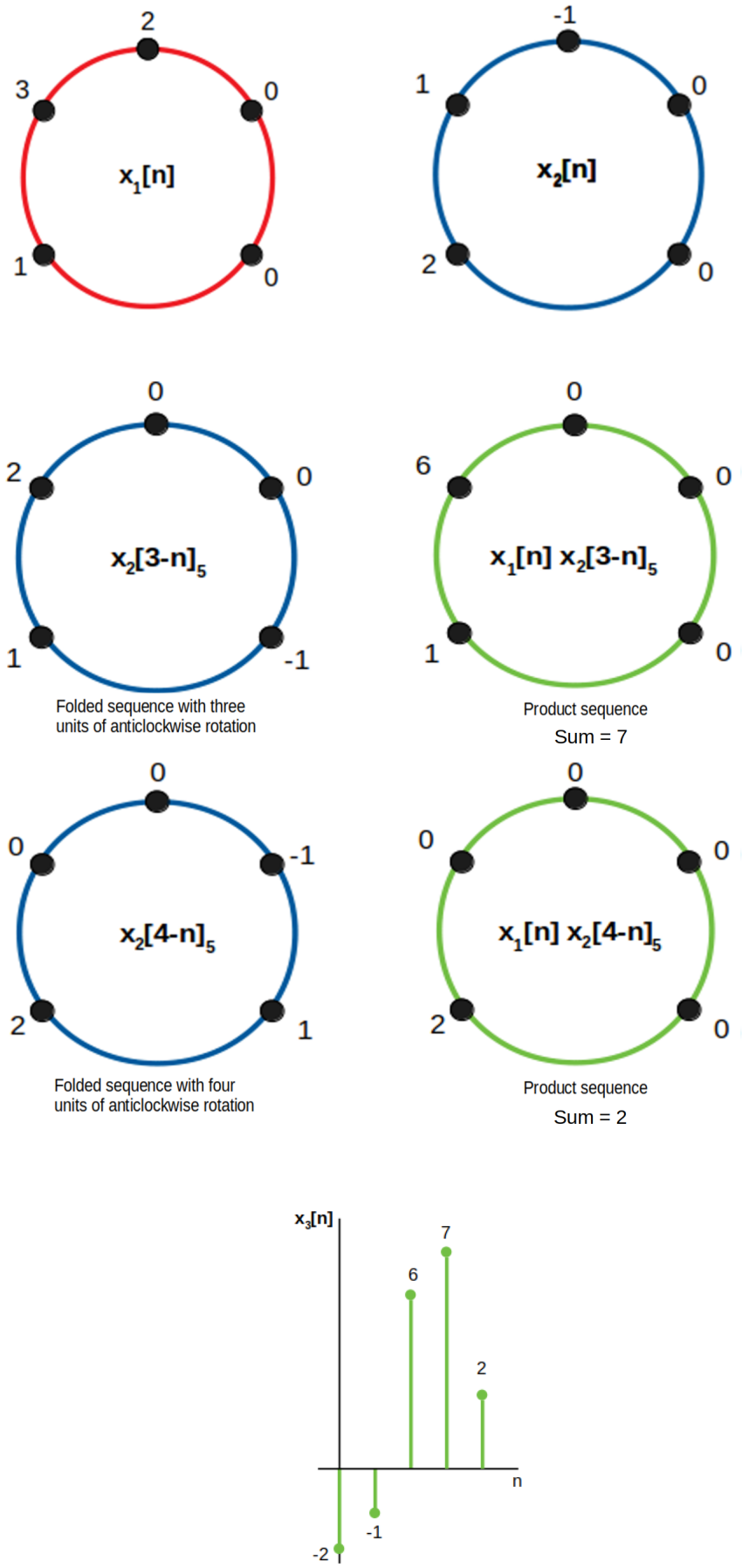


Figure 2: Sequence $x_3[n]$

The above example demonstrated the method of finding linear convolution using circular convolution technique. The next example illustrates the methodology of using DFT and IDFT to find the response of an FIR filter.

Example 1.2 *Determine the response of the FIR filter with impulse response*[1] $h[n] = \{\underset{\uparrow}{1}, 2, 3\}$ *to the input sequence* $x[n] = \{\underset{\uparrow}{1}, 2, 2, 1\}$.

The input sequence has length $L = 4$ *and the impulse response has length* $M = 3$. *Linear convolution of these two sequences produces a sequence of length* $L + M - 1 = 6$. *Hence, the size of the DFTs must be at least* 6.

For simplicity we compute eight-point DFT ($N = 8$). *The reason for choosing an* 8*-point DFT will be clear in Module-12d. The eight point DFTs of* $x[n]$ *and* $h[n]$ *will be given by the following formulas:*

$$X[k] = \sum_{n=0}^{7} x[n] W_8^{kn} \qquad \textit{and} \qquad H[k] = \sum_{n=0}^{7} h[n] W_8^{kn}$$

On computation we get the following: The eight-pont IDFT is

k	$X[k]$	$H[k]$	$Y[k] = X[k]H[k]$
0	6	6	36
1	$\frac{2+\sqrt{2}}{2} - j\frac{4+3\sqrt{2}}{2}$	$1 + \sqrt{2} - j(3 + \sqrt{2})$	$-14.07 - j17.48$
2	$-1 - j$	$-2 - j2$	$j4$
3	$\frac{2-\sqrt{2}}{2} + j\frac{4-3\sqrt{2}}{2}$	$1 - \sqrt{2} + j(3 - \sqrt{2})$	$0.07 + j0.515$
4	0	2	0
5	$\frac{2-\sqrt{2}}{2} - j\frac{4-3\sqrt{2}}{2}$	$1 - \sqrt{2} - j(3 - \sqrt{2})$	$0.07 - j0.515$
6	$-1 + j$	$-2 + j2$	$-j4$
7	$\frac{2+\sqrt{2}}{2} + j\frac{4+3\sqrt{2}}{2}$	$1 + \sqrt{2} + j(3 + \sqrt{2})$	$-14.07 + j17.48$

$$y[n] = \frac{1}{8} \sum_{k=0}^{7} Y[k] W_8^{-kn}, \quad n = 0, 1, \cdots, 7 = \{\underset{\uparrow}{1}, 4, 9, 11, 8, 3, 0, 0\}.$$

Note that the first size values of $y[n]$ *constitutes the desired output values. The last two values are zero because we used an eight-point DFT and IDFT, when, in fact, the minimum number of points required is six.*

What would have happened if instead of using eight-point DFT and IDFT we would have used a four-point DFT and IDFT. Check for yourself what the output would have been then.

[1]The symbol $\uparrow$ is used to denote the zero index of the sequence.

2 Filtering of long data sequence

In practical applications, the input sequence $x[n]$ is often a very long sequence. Since linear filtering performed by DFT involves operations on a block of data, using a long sequence $x[n]$ is not possible due to limited memory of a computer. Hence, the input sequence is divided into blocks and each block is processed via DFT and IDFT to produce a block of output data. These output blocks are fitted together to form an overall output sequence which would be identical to the sequence obtained if the long sequence $x[n]$ had been processed via time-domain convolution.

There are two methods to perform filtering of long data sequence. Both the methods assume the FIR filter has a duration M and the input sequence is segmented into blocks of L-points. By assumption $L >> M$. We discuss the two methods next.

1. **Overlap save method**: In this method

(a) Size of a block is $N = L + M - 1$ (size of $x_1[n]$, $x_2[n]$, etc in Figure 3). DFTs and IDFTs are of length N.

(b) Each data block ($x_1[n]$, $x_2[n]$, etc in Figure 3) consists of the last $M - 1$ data points of the previous data block followed by L new data points from the input $x[n]$ (see Figure 3). The first $M - 1$ points of $x_1[n]$ are zero, since it does not have a previous block.

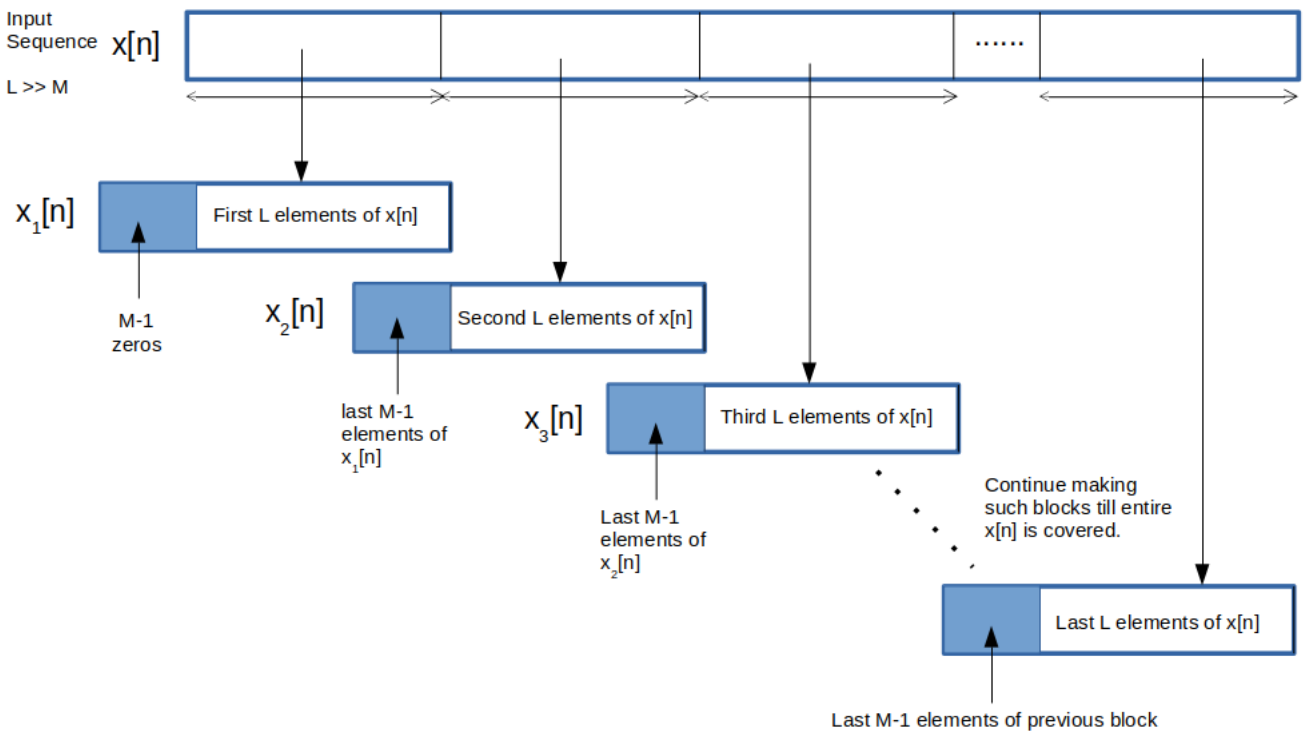


Figure 3: Overlay-save method: formation of input blocks

(c) An N-point DFT of each block, i.e., $x_1[n]$, $x_2[n]$, ... is carried out to give $X_1[k]$, $X_2[k]$, ... as the DFT sequences. (Figure 4)

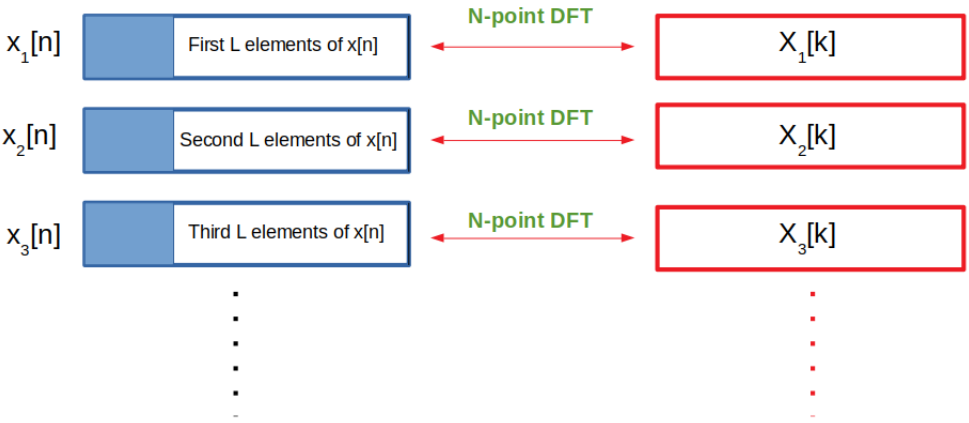


Figure 4: Overlay-save method: N-point DFT of input blocks

(d) The impulse response of the FIR filter is increased in length by appending $L-1$ zeros and an N-point DFT of the sequence $h[n]$ is computed once and stored. (Figure 5)

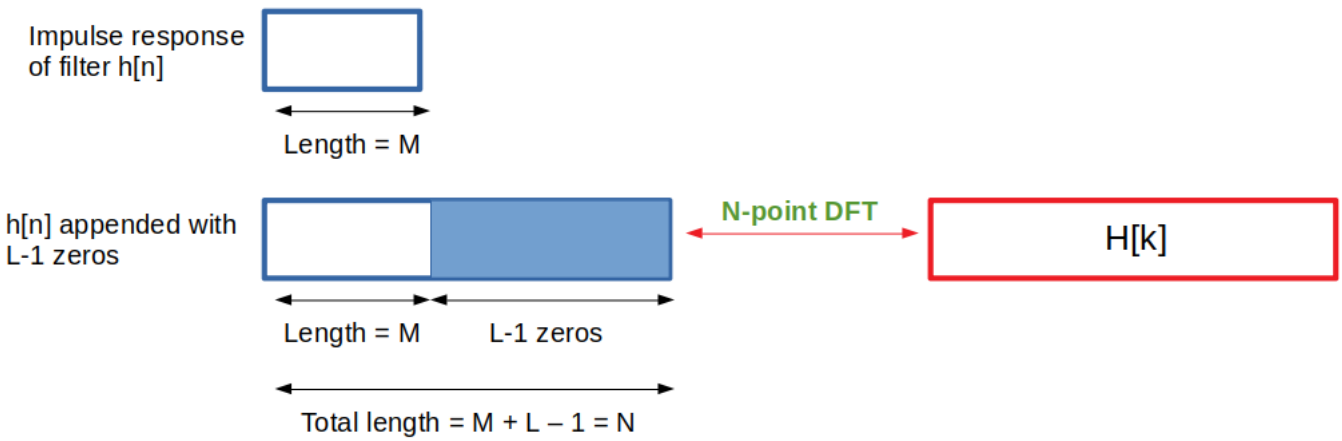


Figure 5: Overlay-save method: N-point DFT of impulse response

(e) The multiplication of the two N-point DFTs $H[k]$ and $X_m[k]$, where $m = 1, 2, \ldots$ yields

$$\hat{Y}_m[k] = H[k]X_m[k] \qquad k = 0, 1, \ldots, N-1.$$

Further, the N-point IDFT of each $Y_m[k]$ yields the following sequence (Figure 6)

$$\hat{y}_m[n] = \begin{bmatrix} \hat{y}_m[0] & \hat{y}_m[1] & \cdots \hat{y}_m[M-1] & \hat{y}_m[M] & \cdots & \hat{y}_m[N-1] \end{bmatrix}$$

(f) Since the data record is of length N, the first $M-1$ points of $\hat{y}_m[n]$ are corrupted by aliasing. We discard the first $M-1$ points of each of the IDFT and then append them together to get the final output sequence $y[n]$. (Figure 7)

Let us look at an example to illustrate the overlap-save method.

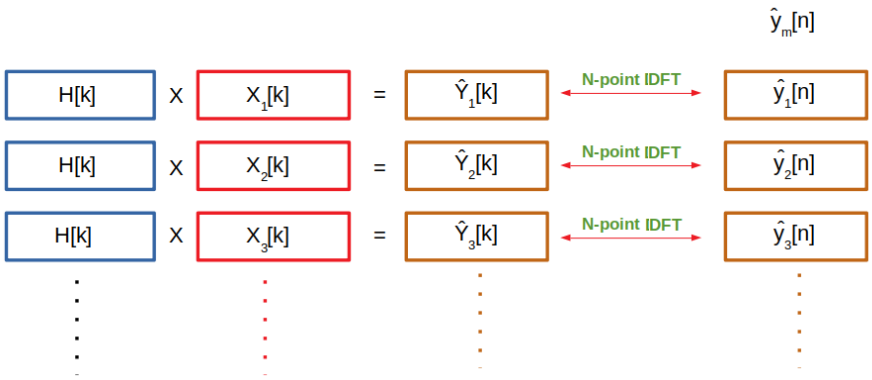


Figure 6: Overlay-save method: Multiplication of N-point DFT

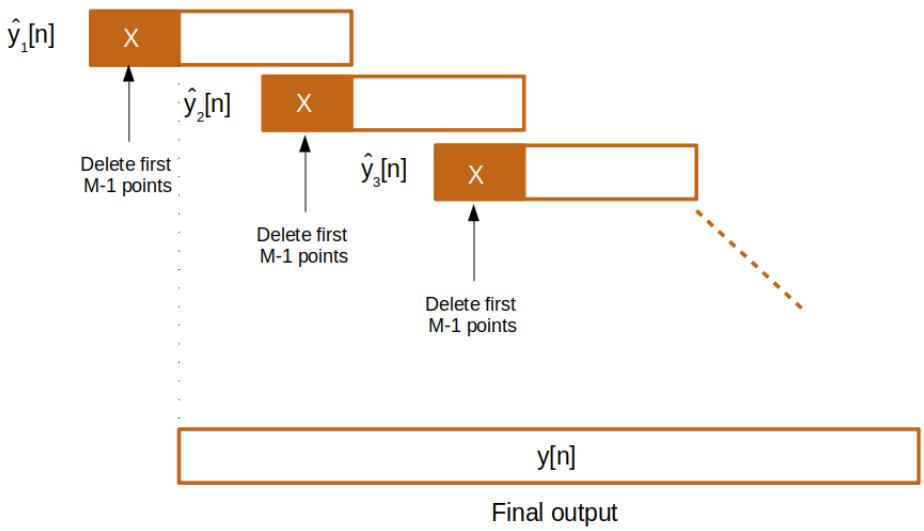


Figure 7: Overlay-save method: Final output

Example 2.1 *Consider* $x[n] = \{3, -1, 0, 1, 3, 2, 0, 1, 2, 1\}$ *and* $h[n] = \{1, 1, 1\}$.

Suppose we take $N = 5$. *Here* $M = 3$ *and therefore,* $N = L + M - 1 \Rightarrow L = 3$.

Hence, $x[n]$ *is divided into blocks as shown in Figure 8 and* $h[n]$ *is augmented with two zeros to make its length* 5.

Note that according to steps (c) and (e) above, we need to find $X_1[k], X_2[k], X_3[k]$, $X_4[k]$, *and then* $H[k]$. *And then we need to find IDFT of* $X_m[k]H[k] = Y_m[k]$ *for* $m = 1, 2, 3, 4$. *In the time-domain this is equivalent to performing circular convolution. Since these are smaller sequences, we can directly find circular convolution of* $x_m[n]$ *and augmented* $h[n]$. *We will get the following:*

Note that in practical cases, Step (c) and (e) are not skipped. Instead the DFT of $y_m[n]$ *and augmented* $h[n]$ *are found. Then IDFT of* $\hat{Y}_m[k]H[k]$ *is performed to compute* $\hat{y}_m[n]$. *Although this is equivalent to circular convolution, since the algorithms to compute DFT (FFT in Module-12d) are faster and efficient, circular convolution is computed using DFT algorithms.*

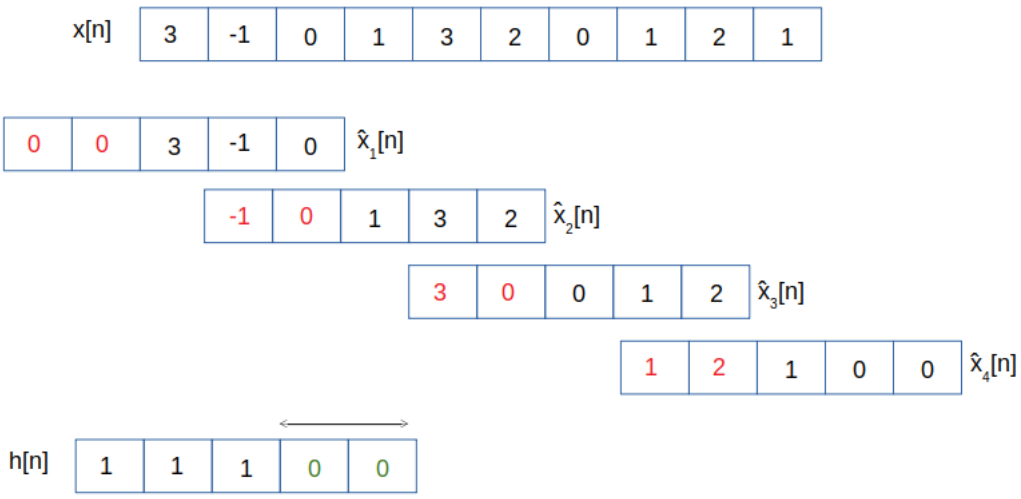


Figure 8: Overlay-save method: Example

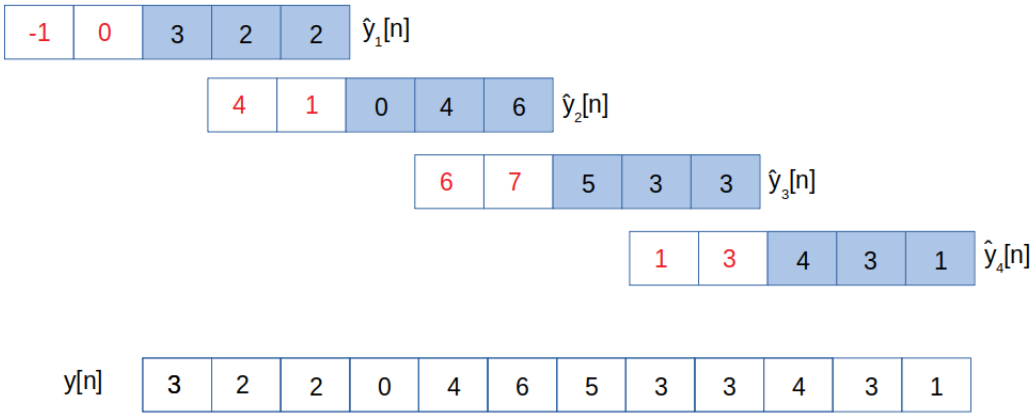


Figure 9: Overlay-save method: Example 2.1 final output

2. **Overlap-add method:** In this method

- Size of a block is $N = L + M - 1$ (size of $x_1[n]$, $x_2[n]$, etc in Figure 3). DFTs and IDFTs are of length N.
- To each data block $x_1[n]$, $x_2[n]$, ... $M - 1$ zeros are appended (see Figure 10).
- An N-point DFT of each block is carried out to give $X_1[k]$, $X_2[k]$, $\cdots$ as the DFT sequences. (see Figure 11)

3. The impulse response of the FIR filter is increased in length by appending $L - 1$ zeros and an N-point DFT of the sequence $H[k]$ is computed once and stored. (Figure 12)

4. The two N-point DFTs are multiplied together to form

$$Y_m[k] = H[k]X_m[k] \qquad k = 0, 1, \ldots, N - 1.$$

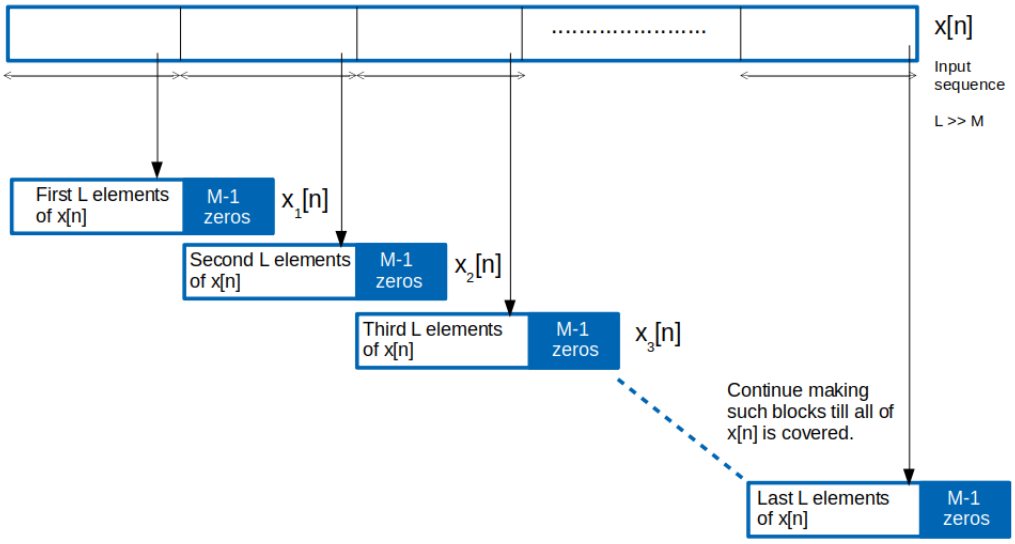


Figure 10: Overlay-add method: formation of input blocks

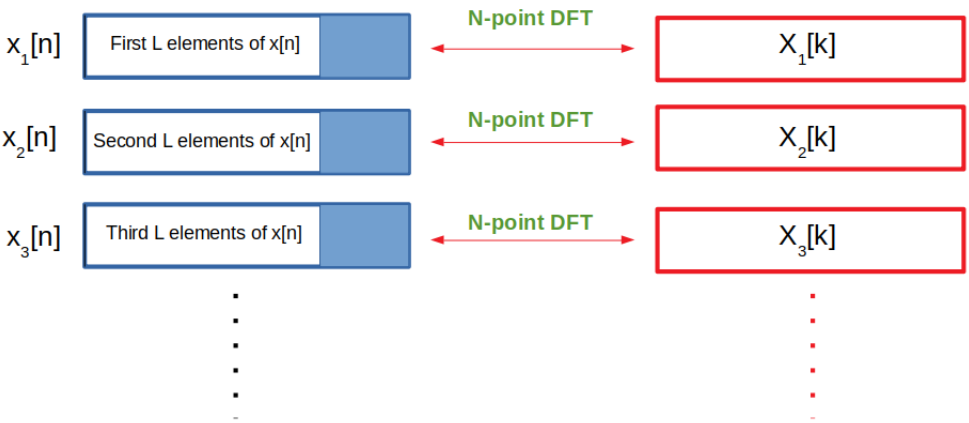


Figure 11: Overlay-add method: N-point DFT of input blocks

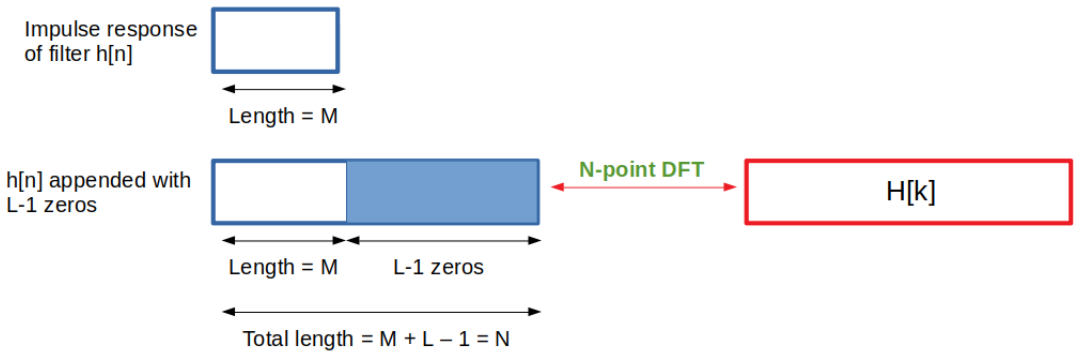


Figure 12: Overlay-add method: N-point DFT of impulse response

Then compute the IDFT of each $Y_m[k]$ to get the sequence $y_m[n]$ for each $m = 1, 2, \ldots$. (Figure 13)

5. Since each data block is terminated with $M-1$ zeros, the last $M-1$ points from each output

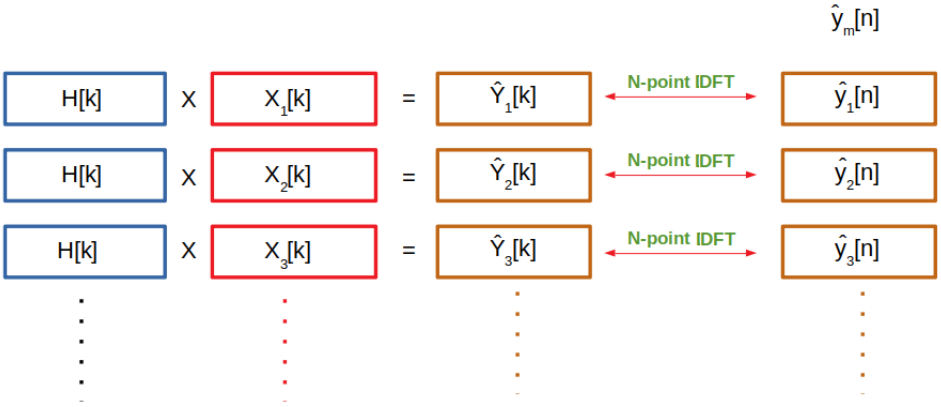


Figure 13: Overlay-add method: Multiplication of N-point DFT

must be overlapped and added to the first $M-1$ points of the succeeding block. Hence, this method is called overlap-add method. This overlapping and adding yields the output sequence as shown in Figure 14.

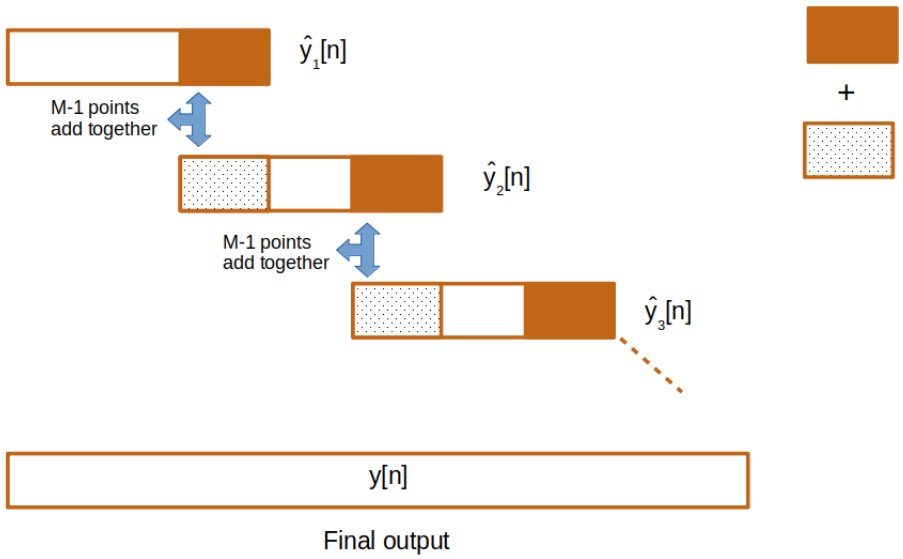


Figure 14: Overlay-add method: Final output